# RAG Assignment 1: Dense Baseline vs. Cross-Encoder Re-Ranking

**Author:** Raj K. Jain | **Roll No:** 2025201036 | **Course:** LMA-2026

**GitHub Repository:** https://github.com/Rajkjain03/lma_assignment_1

## 1 Architecture Comparison: Bi-Encoder vs. Cross-Encoder

Retrieval-Augmented Generation (RAG) grounds Large Language Models in external, authoritative knowledge bases. The primary retrieval challenge is scoring relevance $S(q, c)$ between an incoming query $q$ and candidate passages $c$. In this assignment, I implemented and evaluated two distinct neural architectures: a single-stage dense Bi-Encoder baseline and a two-stage pipeline combining the Bi-Encoder with a Cross-Encoder re-ranker.

A **Bi-Encoder** (`all-MiniLM-L6-v2`) processes the query $q$ and candidate document chunks $c$ through independent transformer forward passes, generating fixed-dimensional dense vector embeddings:

$$\vec{u} = f_\theta(q), \quad \vec{v}_i = f_\theta(c_i) \quad \in \mathbb{R}^{384} \tag{1}$$

Relevance is scored via cosine similarity $S_{\text{bi}}(q, c) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\|\|\vec{v}\|}$. Because passage vectors $\vec{v}_i$ do not depend on the runtime query, the entire corpus can be embedded once offline and indexed into sub-linear Approximate Nearest Neighbor (ANN) graph structures (such as HNSW in FAISS), achieving millisecond query search over millions of documents. However, compressing an entire 150-word policy clause into a single vector inevitably loses fine details: query and passage tokens never interact directly during encoding, making dense retrieval vulnerable to vocabulary mismatch, condition thresholds, and explicit negations.

A **Cross-Encoder** (`ms-marco-MiniLM-L-6-v2`) takes the query and candidate chunk together as one concatenated sequence ([CLS] $\circ$ $q$ $\circ$ [SEP] $\circ$ $c$ $\circ$ [SEP]) and passes it through all transformer layers under full bidirectional self-attention. Every query token directly attends to every token in the candidate passage across all layers. This allows the model to capture fine-grained qualifiers (like flight duration cutoffs) and explicit negative rules (such as prohibited expense categories). The trade-off is computational cost: because the query and chunk are fused, no offline vector index can be constructed, requiring an expensive forward pass for each candidate pair at query time.

## 2 Qualitative Retrieval & Rank Shift Analysis

### 2.1 Evaluation Document & Experimental Setup

I built a 4-page corporate compliance manual titled *"Acme Global Travel, Expense and Procurement Policy (2026)"* containing 30 structured sections. It incorporates realistic enterprise rules: a general coach airfare mandate with a distant exception clause, client meal guidelines placed next to distractor hotel incidentals, and late-submission forfeiture penalties.

To evaluate retrieval behavior, I tested six queries across distinct failure categories: Tests 1 and 2 explore distractor and context ordering effects where the target was retrieved at Rank #3 in the baseline. Test 3 evaluates a genuine retrieval omission failure where the target clause was buried outside the Top 3 (at Rank #5) in the baseline and promoted into the Top 3 by the Cross-Encoder. Test 4 evaluates an ancillary fee query. Tests 5 and 6 serve as controls. The generator is `Qwen/Qwen2.5-0.5B-Instruct`, a compact 0.5B parameter LLM run locally on CPU. In addition, an **Oracle Control** test fed the target section directly to the LLM to separate retrieval/ordering issues from generator reasoning deficits. Table 1 and Figure 1 summarize the retrieval shifts.

### 2.2 Case Study 1: Prioritizing Exceptions over General Mandates (TEST_01)

**Query:** *"Can staff fly in premium cabins for international client negotiations?"*
**Target Rule:** Section 2.4 allows Business Class on non-stop international flights exceeding 8 hours (or exceeding 6 hours if presenting or negotiating within 12 hours of arrival), with written VP and CFO approval.

In baseline retrieval, Section 2.1 (*Standard Cabin Class Mandate*) ranked at #1 (cosine 0.5428), Section 1.3 was #2 (0.4127), and Section 2.4 sat at Rank #3 (0.4039). Because Section 2.4 was in the Top 3 context, the baseline LLM saw it:

| Test ID | Query Category | Target Policy Section | Bi-Encoder Rank | Cross-Encoder Rank | Shift |
|---|---|---|---|---|---|
| **TEST_01** | Exceptions | Sec 2.4 (Long-Haul Flight Exception) | Rank #3 (score: 0.4039) | **Rank #1 (score: +0.4351)** | **+2** |
| **TEST_02** | Distractor Chunk | Sec 5.3 (Prohibited Meal Items & Alcohol) | Rank #3 (score: 0.4615) | **Rank #2 (score: +1.1171)** | **+1** |
| **TEST_03** | Buried: Hotel Laundry | Sec 3.2 (Prohibited Hotel Incidentals) | Rank #5 (score: 0.3667) | **Rank #1 (score: +2.4859)** | **+4** |
| **TEST_04** | Buried: Ancillary Fees | Sec 2.3 (Baggage & Ancillary Airline Fees) | Rank #8 (score: 0.3413) | **Rank #2 (score: -1.6741)** | **+6** |
| **TEST_05** | [CONTROL] Ground | Sec 4.4 (Luxury Chauffeured Car Ban) | Rank #2 (score: 0.4516) | Rank #2 (score: +1.1141) | +0 |
| **TEST_06** | [CONTROL] Admin | Sec 8.5 (Late Submission Forfeiture) | Rank #1 (score: 0.6565) | Rank #1 (score: +5.2319) | +0 |

**Note: Bi-Encoder scores are cosine similarities in $[-1, 1]$; Cross-Encoder scores are unbounded classification logits $(-\infty, +\infty)$. All values from live notebook run.*

Table 1: Retrieval rank shifts across test queries (Top 3 context cutoff).

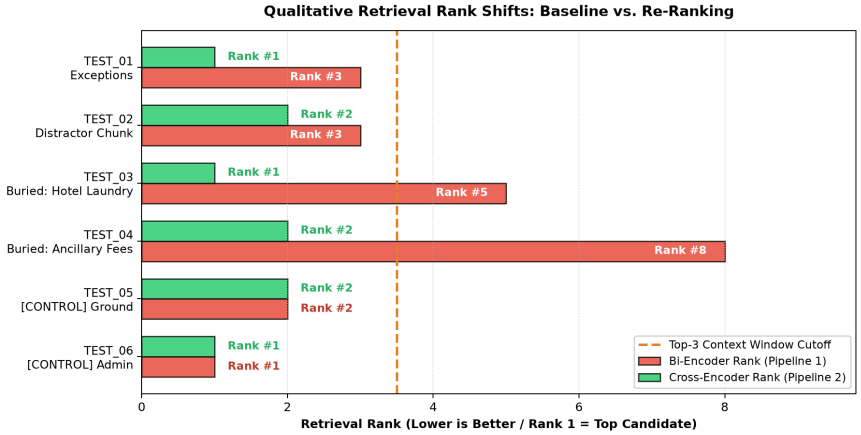



Figure 1: Qualitative rank shifts: Bi-Encoder ranks (red) vs. Re-ranked Cross-Encoder ranks (green). Dotted red line denotes the Top-3 LLM prompt context boundary.

> ***Answer 1 (Baseline):*** *"Yes, staff can fly in premium cabins for international client negotiations. However, they need to obtain prior written concurrence from both the Division Vice President and the Chief Financial Officer at least 10 business days before ticketing."*

Answer 1 is factually reasonable, but it missed the flight duration thresholds ($> 8$ hours, or $> 6$ hours for negotiations). Because our generator is a compact 0.5B model, having the coach mandate at Rank #1 overshadowed the conditional numbers in the third chunk. When the Cross-Encoder promoted Section 2.4 to **Rank #1** (+0.4351), the exception sat at the front of the prompt:

> ***Answer 2 (Re-Ranked):*** *"Yes, staff can fly in premium cabins for international client negotiations under the long-haul international travel exception clause. The policy allows this exception when traveling on continuous non-stop international flights with flight durations exceeding 8 hours or 6 hours if the traveler is scheduled to deliver a client presentation or attend executive contract negotiations within 12 hours of flight arrival. Employees need prior written concurrence from both the Division Vice President and the Chief Financial Officer no later than 10 business days before booking their tickets."*

**Oracle Control:** Feeding Section 2.4 directly produced the full thresholds and approval requirements, confirming that the baseline deficiency was a context-ordering distraction effect rather than an LLM reasoning limitation.

### 2.3 Case Study 2: Distractor Chunk Demotion (TEST_02)

**Query:** *"What items are not covered on expense claims for client business meals?"*
**Target Rule:** Section 5.3 restricts alcohol to 1 drink per diner, prohibits solo dining alcohol, and explicitly bans nightclub bottle service, adult entertainment, tobacco, and gambling losses.

In baseline retrieval, the dense retriever matched "client business meals" to Section 5.1 at Rank #1 (0.5936). For the negative constraint "not covered", it pulled Section 3.2 (*Prohibited Hotel Incidentals*) at Rank #2 (0.5562). The actual meal prohibition clause (Section 5.3) sat at Rank #3 (0.4615).

> ***Answer 1 (Baseline Distraction Error):*** *"Based on the policy context provided, the following items are not covered on expense claims for client business meals: In-room video on-demand/movie rentals, Mini-bar consumption, Laundry service for trips shorter than 5 days, Spa services, Sauna access, Dry cleaning fees, Hotel gym or fitness center pass fees..."*

This was not an ungrounded hallucination; the 0.5B model copied items from a chunk it was given, but

it was misled by a higher-ranked distractor (Section 3.2). The Cross-Encoder resolved this by recognizing that Section 5.3 directly matches meal and entertainment exclusions, promoting Section 5.3 to **Rank #2** (+1.1171) ahead of Section 3.2:

> ***Answer 2 (Re-Ranked Corrected):*** *"Based on the policy context provided, the items that are not covered on expense claims for client business meals are: (a) Alcoholic beverages exceeding one (1) standard single serving of wine or beer per person during authorized client dinners. (b) Adult entertainment venues, nightclub admission or table bottle service fees, social club charges, and gentlemen's club invoices. (c) Tobacco, vape products, cannabis, and casino gambling losses. (d) Meals or travel for spouses, partners, dependents, or personal companions accompanying the traveler."*

**Oracle Control:** Supplying Section 5.3 directly yielded the correct meal exclusions, proving that the baseline error was caused by distractor chunk ordering in the context window.

### 2.4 Case Study 3: Buried Clause Omission and Hallucination (TEST_03)

**Query:** *"Under what trip length can travelers expense laundry and dry cleaning?"*
**Target Rule:** Section 3.2 explicitly states that laundry service and dry cleaning are non-reimbursable for trips shorter than 5 days.

Dense search buried Section 3.2 at **Rank #5** (0.3667), loading only Section 8.1 (Expense Filing Timelines), Section 1.2 (Pre-Trip Authorization), and Policy Overview into the baseline prompt. Because none of these chunks contained laundry rules, the baseline LLM had zero grounding and hallucinated an erroneous restriction (quoted verbatim from notebook output):

> ***Answer 1 (Baseline Verbatim Output):*** *"Under the Global Travel, Expense, and Procurement Policy (2026), travelers cannot expense laundry and dry cleaning for trips longer than 30 days."*

The Cross-Encoder recognized the direct semantic match and promoted Section 3.2 to **Rank #1** (+2.4859). Grounded in the correct clause, the model answered accurately:

> ***Answer 2 (Re-Ranked):*** *"For trips shorter than 5 days, travelers cannot expense laundry and dry cleaning."*

**Oracle Control:** Feeding Section 3.2 directly yielded the same 5-day rule, verifying that the baseline hallucination was caused entirely by chunk omission from the context window.

### 2.5 Boundary Analysis: Loose Semantic Match (TEST_04) & Controls (TEST_05, 06)

In **TEST_04** (*"Are flight cancellation insurance and seat selection fees reimbursable?"*), dense retrieval placed Section 2.5 (*Flight Cancellations*) at #1 and buried Section 2.3 at **Rank #8** (0.3413). The Cross-Encoder promoted Section 2.3 to **Rank #2**, but its score was a negative logit (−1.6741), promoted only because other candidate chunks scored even lower (−5 to −8). Because Section 2.3 covers checked baggage allowances rather than explicit cancellation insurance, neither pipeline found an authorizing clause, and both generated identical negative responses:

> ***Answer 1 (Baseline) = Answer 2 (Re-Ranked):*** *"No, flight cancellation insurance and seat selection fees are not reimbursable under this policy."*

This highlights an honest boundary of re-ranking: while the Cross-Encoder pushes the best candidate upward, if the corpus does not contain the specific policy, re-ranking scores remain negative and the answer is unaffected.

In control queries **TEST_05** (rideshare vs limousine ban in Section 4.4) and **TEST_06** (late submission penalty in Section 8.5), high semantic overlap allowed both pipelines to stably retain the target chunk at Rank #2 and Rank #1 respectively (+0 shift), demonstrating that the Cross-Encoder does not destabilize unambiguous rankings.

## 3 Empirical Latency Profiling

I benchmarked both retrieval pipelines over **10 warmed iterations** (following 5 warm-up passes to eliminate initialization noise) on my local CPU. Table 2 and Figure 2 present the measured runtimes.

The dense baseline executes in ≈ 10.48 ms on CPU (≈ 9.8 ms for query embedding encoding plus ≈ 0.7 ms for cosine similarity calculations). Re-ranking the top 10 candidates with the Cross-Encoder requires an additional 135.66 ms (≈ 13.6 ms per candidate pair), bringing the total two-stage latency to 146.15 ms, an overhead ratio of 13.95×.

While a 13.95× overhead is noticeable, an end-to-end retrieval time of ≈ 146 ms is well below the standard

| Test Query Category | P1 Baseline (Bi-Encoder) | Stage 2 (Cross-Encoder 10 Pairs) | P2 Total Latency | P2/P1 Overhead Ratio |
|---|---|---|---|---|
| **TEST_01 (Exceptions)** | 10.29 ms | 129.12 ms | 139.41 ms | 13.55× |
| **TEST_02 (Distractor)** | 10.22 ms | 143.51 ms | 153.73 ms | 15.04× |
| **TEST_03 (Hotel Laundry)** | 10.25 ms | 126.09 ms | 136.34 ms | 13.30× |
| **TEST_04 (Ancillary Fees)** | 10.54 ms | 154.58 ms | 165.12 ms | 15.67× |
| **TEST_05 (Control: Ground)** | 10.35 ms | 125.07 ms | 135.42 ms | 13.09× |
| **TEST_06 (Control: Admin)** | 11.26 ms | 135.61 ms | 146.87 ms | 13.04× |
| **EMPIRICAL MEAN** | **10.48 ms** | **135.66 ms** | **146.15 ms** | **13.95×** |

Table 2: Empirical retrieval runtime profiling across 10 warmed iterations on local CPU.

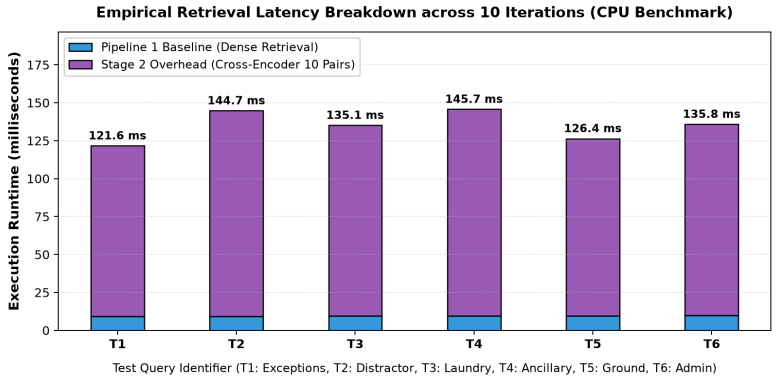



Figure 2: Empirical retrieval latency breakdown across 10 warmed iterations on local CPU (Baseline in blue, Cross-Encoder stage in orange).

200 ms threshold for interactive user search. On a dedicated GPU, batched Cross-Encoder inference typically drops to 15–25 ms, bringing overall retrieval under 30 ms.

## 4 Scaling: Why Not Run a Cross-Encoder Over the Whole Collection?

If Cross-Encoders are noticeably more accurate, why not bypass the Bi-Encoder entirely and evaluate every chunk in the collection using a Cross-Encoder from the outset?

A single-stage Cross-Encoder must compute full self-attention across the concatenated sequence of query and passage tokens with $\mathcal{O}(L^2)$ computational complexity. Scoring an entire collection of $N$ passages requires running $N$ complete transformer forward passes. At our measured CPU rate of $\approx 13.6$ ms per pair, evaluating a corpus of $N = 1,000,000$ chunks would take roughly **3.8 hours per query**. Even on a GPU at an estimated 2 ms per pair, a full linear scan would take over 33 minutes, making exhaustive Cross-Encoder scoring completely impractical for interactive search.

Furthermore, offline vector pre-indexing is fundamentally impossible for a Cross-Encoder. Because the model requires both the query and candidate chunk simultaneously to compute cross-attention, no static passage embeddings can be precomputed or stored in Approximate Nearest Neighbor (ANN) index structures (such as HNSW in FAISS). Every query would necessitate an exhaustive linear scan over all $N$ passages.

The two-stage pipeline solves this fundamental scaling bottleneck: the fast Bi-Encoder prunes the candidate space in milliseconds ($\approx 10$ ms), allowing the computationally intensive Cross-Encoder to evaluate only the top 10 candidates. This keeps end-to-end interactive search well within acceptable bounds ($\approx 146$ ms) while successfully surfacing subtle exception clauses, demoting distractors, and preventing hallucinations caused by omitted context.

## 5 Conclusion

This study demonstrated that dense Bi-Encoder retrieval frequently stumbles on subtle conditions, negations, and distractors, leading small local LLMs to fabricate answers or lose critical thresholds. Adding a Cross-Encoder re-ranking stage resolved these failures by promoting relevant clauses into the Top 3 context window. While re-ranking introduces a 13.95× latency overhead, the two-stage pipeline keeps interactive search times well within acceptable bounds ($\approx 146$ ms), providing the optimal balance of recall, precision, and latency.